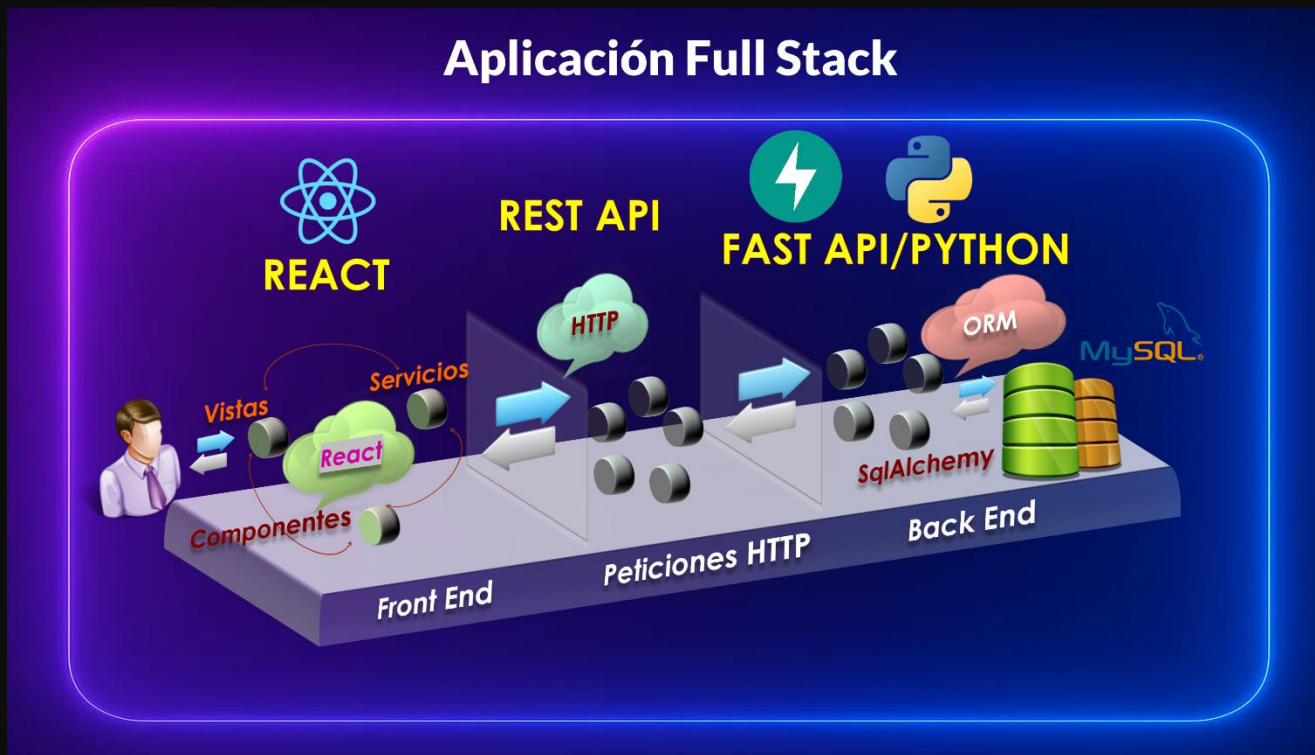




🔧 FASE 1: CREACIÓN DEL PROYECTO Y ENTORNO VIRTUAL

Por qué:

Antes de escribir una sola línea de código, es clave tener un proyecto ordenado y un entorno aislado. Eso nos permite:

- Mantener las dependencias de este proyecto separadas de otros.
- Controlar fácilmente qué versión de cada librería estamos usando.
- Preparar una base limpia para todo el backend de la Biblioteca Personal.

FastAPI (visión rápida):

FastAPI es un framework moderno para construir APIs en Python, muy rápido y basado en *type hints*. Sus ventajas principales:

- Genera documentación automática (*/docs*).
- Valida datos de entrada usando modelos (Pydantic).
- Es muy eficiente gracias a ASGI y a Uvicorn.

Entorno virtual (visión rápida):

Un entorno virtual (*virtualenv*) es una carpeta que contiene su propio intérprete de Python y sus propias librerías. Así, lo que instalamos para este proyecto no afecta a otros proyectos de Python en la misma máquina.

📁 NUEVOS ARCHIVOS / MODIFICACIONES (Fase 1)

```
C:\
├─ Cursos\
│   └─ FastAPI\
│       └─ biblioteca_fastapi\ (NUEVO, carpeta del proyecto)
│           └─ .venv\ (NUEVO, entorno virtual de Python)
```

En esta fase **solo** creamos la carpeta del proyecto y el entorno virtual.
No habrá todavía archivos de código del backend (`main.py`, `database.py`, etc.).

1 Crear carpeta base del proyecto

Ruta objetivo del proyecto:

- C:\Cursos\FastAPI\biblioteca_fastapi\

En **CMD** o **PowerShell**, ejecuta:

```
mkdir C:\Cursos\FastAPI
mkdir C:\Cursos\FastAPI\biblioteca_fastapi
cd C:\Cursos\FastAPI\biblioteca_fastapi
```

Con esto dejamos lista la carpeta:

- C:\Cursos\FastAPI\biblioteca_fastapi\ 📁 carpeta raíz del backend.
-

2 Crear el entorno virtual dentro del proyecto

Desde la carpeta del proyecto:

- C:\Cursos\FastAPI\biblioteca_fastapi\

ejecuta:

```
python -m venv .venv
```

Esto creará la carpeta:

- C:\Cursos\FastAPI\biblioteca_fastapi\.venv\

Ahí vivirán:

- El intérprete de Python dedicado al proyecto.
 - Todas las librerías que instalemos para este backend.
-

3 Activar el entorno virtual

Asegúrate de seguir en:

- `C:\Cursos\FastAPI\biblioteca_fastapi\`

En PowerShell (recomendado):

```
.\.venv\Scripts\Activate.ps1
```

En CMD clásico:

```
.\.venv\Scripts\activate.bat
```

Si la activación fue correcta, deberías ver algo parecido a:

```
(.venv) C:\Cursos\FastAPI\biblioteca_fastapi>
```

Eso indica que todo lo que instales con `pip` se quedará dentro de `.venv` y no afectará al resto del sistema.

4 Instalar las dependencias mínimas del proyecto

Con el entorno virtual **activo**, instala las librerías base:

```
pip install fastapi "uvicorn[standard]" sqlalchemy alembic python-dotenv pymysql cryptography
```

Paquetes que estás instalando:

- `fastapi` → framework principal de la API.
 - `uvicorn[standard]` → servidor ASGI para ejecutar FastAPI.
 - `sqlalchemy` → ORM para conectarnos a MySQL de forma elegante.
 - `alembic` → gestiona migraciones de base de datos.
 - `python-dotenv` → carga variables desde un archivo `.env`.
 - `pymysql` → conector para MySQL.
 - `cryptography` → requerido por algunos componentes (útil para conexiones seguras, etc.).
-

5 Verificar instalación de dependencias

Puedes comprobar que todo se instaló bien con:

```
pip list
```

Deberías ver en la lista paquetes como:

- fastapi
- uvicorn
- SQLAlchemy
- alembic
- python-dotenv
- PyMySQL
- cryptography

Opcionalmente, una prueba rápida:

```
python -c "import fastapi, sqlalchemy, alembic, dotenv, pymysql; print('Dependencias OK')"
```

Si imprime `Dependencias OK` sin errores, las librerías están disponibles en el entorno virtual.

6 Asociar el entorno virtual en PyCharm (opcional pero recomendado)

En PyCharm:

- Abre el folder: `C:\Cursos\FastAPI\biblioteca_fastapi\`.
- Ve a:
`File > Settings > Project: biblioteca_fastapi > Python Interpreter`
- Elige **Existing environment** (o similar) y selecciona:
 - o `C:\Cursos\FastAPI\biblioteca_fastapi\.venv\Scripts\python.exe`

Con esto te aseguras de que PyCharm use el mismo entorno virtual que configuraste en consola.

➡ En resumen, al terminar esta fase tienes:

- Carpeta del proyecto creada.
- Entorno virtual `.venv` listo y activable.
- Dependencias principales instaladas.

Aún **no** hay archivos Python del backend (eso iniciará en la siguiente fase con `main.py`).

¿Continuamos? Responde: **listo** para la Fase 2 (estructura base y primera app).

Dijiste:

listo vamos a la fase 2

ChatGPT dijo:

🌸 FASE 2: ESTRUCTURA BASE Y APLICACIÓN FASTAPI INICIAL

Por qué:

Antes de crear modelos, BD o servicios, necesitamos el **punto de entrada** del backend: un archivo `main.py` que inicialice la aplicación FastAPI y exponga un endpoint mínimo que confirme que la API está funcionando.

Esto nos permite:

- Verificar que FastAPI está bien instalado.
 - Confirmar que Uvicorn ejecuta correctamente el servidor.
 - Probar desde el navegador o Postman que el proyecto responde.
-

📁 NUEVOS ARCHIVOS / MODIFICACIONES (Fase 2)

```
biblioteca_fastapi/  
└─ main.py    (NUEVO)
```

En esta fase solo aparece un archivo nuevo: `main.py`.



Archivo (NUEVO): `C:\Cursos\FastAPI\biblioteca_fastapi\main.py`

A continuación tienes el archivo **completo**, listo para copiar y pegar:

```
# main.py  
# Punto de entrada de la aplicación FastAPI.  
  
from fastapi import FastAPI  
  
# Crear instancia principal de la aplicación FastAPI.  
# 'app' será utilizada por Uvicorn para levantar el servidor.  
app = FastAPI()
```

```
    title="API Biblioteca Personal",
    description="Backend REST con FastAPI para gestionar libros.",
    version="1.0.0"
)

# Endpoint de prueba (ruta raíz).
# Sirve para verificar que la API está viva.
@app.get("/")
def raiz():
    """
    Endpoint básico que confirma que la API está funcionando.
    """
    return {"mensaje": "API Biblioteca funcionando correctamente"}
```

Ejecutar la app en el puerto por defecto (8000)

En consola, dentro de la carpeta del proyecto:

```
uvicorn main:app --reload
```

- main → nombre del archivo (main.py)
- app → nombre de la instancia de FastAPI creada arriba
- --reload → reinicia automáticamente cuando detecta cambios

Si todo está bien, verás algo como:

```
Uvicorn running on http://127.0.0.1:8000
```

Probar la API

✓ Desde navegador:

Entra a:

- <http://localhost:8000/>

Deberías ver:

```
{"mensaje": "API Biblioteca funcionando correctamente"}
```

✓ Documentación automática:

FastAPI genera documentación estilo Swagger en:

- `http://localhost:8000/docs`

Aquí podrás ver todos los endpoints que creemos más adelante.

✓ Desde Postman o curl:

```
curl http://localhost:8000/
```

Con esto la API está *oficialmente viva* y lista para crecer.

¿Continuamos? Responde: **listo** para la Fase 3 (conexión con MySQL y `.env`).

Dijiste:

listo vamos a la fase 3

ChatGPT dijo:

✿ FASE 3: CONEXIÓN CON MYSQL Y ARCHIVO `.env`

Por qué:

Antes de crear modelos o migraciones necesitamos una **conexión real** hacia MySQL.

En esta fase vamos a:

- Crear la base de datos.
- Crear el archivo `.env` con las variables de conexión.
- Construir `database.py`, el módulo central para conectarse a MySQL.
- Probar que la conexión funciona ejecutando `python database.py`.

Con esto dejaremos lista toda la infraestructura para que Alembic y SQLAlchemy funcionen correctamente en las siguientes fases.

📁 NUEVOS ARCHIVOS / MODIFICACIONES (Fase 3)

```
biblioteca_fastapi/  
├── .env                (NUEVO)  
├── database.py         (NUEVO)  
└── main.py
```

1 Crear la base de datos en MySQL

Abre **MySQL Workbench** o tu cliente preferido y ejecuta:

```
CREATE DATABASE biblioteca_db;
```

2 Archivo (NUEVO): .env

Crea el archivo:

- C:\Cursos\FastAPI\biblioteca_fastapi\.env

Contenido **completo** (listo para copiar y pegar):

```
# Variables de conexión a MySQL
DB_USER=root
DB_PASSWORD=admin
DB_HOST=localhost
DB_PORT=3306
DB_NAME=biblioteca_db
```

Este archivo **no se sube a repositorios públicos**.
FastAPI usará estas variables para construir la URL de conexión.

3 Archivo (NUEVO): database.py

Crea:

- C:\Cursos\FastAPI\biblioteca_fastapi\database.py

A continuación el archivo **COMPLETO**, listo para copiar y pegar:

```
# database.py
# Configuración de la base de datos, sesión y prueba de conexión.

import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from dotenv import load_dotenv

# Cargar las variables del archivo .env
load_dotenv()

# Construcción de la URL de conexión a MySQL
DB_USER = os.getenv("DB_USER")
```



```

DB_PASSWORD = os.getenv("DB_PASSWORD")
DB_HOST = os.getenv("DB_HOST")
DB_PORT = os.getenv("DB_PORT")
DB_NAME = os.getenv("DB_NAME")

DATABASE_URL = (
    f"mysql+pymysql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
)

# Creación del motor de conexión
engine = create_engine(
    DATABASE_URL,
    echo=False # Cambiar a True si quieres ver las consultas SQL
)

# Crear un SessionLocal por petición
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Clase base para los modelos
Base = declarative_base()

# Dependencia para FastAPI: obtiene una sesión por petición y la cierra
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Prueba rápida de conexión: ejecutar con "python database.py"
if __name__ == "__main__":
    try:
        with engine.connect() as conn:
            print("✓ Conexión exitosa a MySQL")
    except Exception as e:
        print("✗ Error al conectar a la base de datos:", e)

```



Explicación breve del archivo `database.py`

- **load_dotenv()** → carga las credenciales desde `.env`.
- **DATABASE_URL** → arma la cadena usada por SQLAlchemy para conectarse a MySQL.
- **engine** → conexión principal hacia la BD.
- **SessionLocal** → cada petición HTTP abrirá su propia sesión.
- **Base** → clase que usarán todos los modelos (como `Libro`).
- **get_db()** → se inyectará en los endpoints (`Depends(get_db)`).
- Bloque `if __name__ == "__main__":` → permite ejecutar `python database.py` y confirmar si MySQL responde correctamente.



Probar la conexión

Ejecuta:

```
python database.py
```

Si todo está bien:

✓ Conexión exitosa a MySQL

Si hay algún error (contraseña, puerto, BD inexistente), el mensaje te lo indicará.

Con esto ya tenemos la infraestructura segura y profesional para manejar la base de datos.

¿Continuamos? Responde: **listo** para la Fase 4 (modelo Libro y configuración de Alembic).

Dijiste:

listo vamos a la fase 4

ChatGPT dijo:



FASE 4: MODELO `Libro` EN CARPETA `models`

Por qué:

Ya tenemos la conexión a MySQL y el entorno listo. Ahora necesitamos **modelar la tabla `libros`** usando SQLAlchemy.

El modelo será la representación en Python de la tabla real en MySQL, y Alembic lo usará para generar las migraciones.

En esta fase:

- Creamos la carpeta `models`.
 - Creamos el archivo `libro.py`.
 - Definimos la clase `Libro` con sus columnas y tipos correctos.
-



NUEVOS ARCHIVOS / MODIFICACIONES (Fase 4)

```
biblioteca_fastapi/  
|
```

```

├── models/                (NUEVO)
│   └── libro.py           (NUEVO)
├── database.py
└── main.py

```



Archivo (NUEVO): `models/libro.py`

Ruta exacta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\models\libro.py`

A continuación el archivo **completo**, listo para copiar y pegar:

```

# models/libro.py
# Definición del modelo Libro para la base de datos.

from sqlalchemy import Column, Integer, String
from database import Base

class Libro(Base):
    """
    Modelo SQLAlchemy que representa la tabla 'libros'.
    Cada atributo corresponde a una columna en la base de datos.
    """
    __tablename__ = "libros"

    # ID autoincremental y llave primaria
    id = Column(Integer, primary_key=True, autoincrement=True)

    # Título del libro (cadena obligatoria)
    titulo = Column(String(255), nullable=False)

    # Autor del libro (cadena obligatoria)
    autor = Column(String(255), nullable=False)

    # Rating entre 1 y 5 (entero obligatorio)
    rating = Column(Integer, nullable=False)

```



Explicación del modelo

- `__tablename__`
Define el nombre exacto de la tabla en la base de datos: `libros`.
- `id`
 - Entero.
 - Llave primaria.
 - Autoincremental.
- `titulo y autor`
 - Tipo `String(255)` adecuado para texto corto.

- `nullable=False` → obligatorios.
 - **rating**
 - Tipo entero.
 - Validaremos su rango posteriormente con Pydantic.
-

Organización recomendada

- Carpeta `models` → contiene *todos* los modelos del proyecto.
Esto mantiene limpio el backend y facilita la detección de modelos por Alembic.
-

¿Continuamos? Responde: **listo** para la Fase 5 (iniciar Alembic + generar migraciones).

Dijiste:

listo vamos a la fase 5

ChatGPT dijo:

FASE 5: INICIAR ALEMBIC + GENERAR MIGRACIONES

Por qué:

Ahora que ya tenemos el modelo `Libro`, necesitamos que esa definición se transforme en una tabla real dentro de MySQL.

Para eso usamos **Alembic**, el sistema oficial de migraciones de SQLAlchemy.

Alembic:

- Detecta cambios en modelos.
- Versiona la estructura de la BD.
- Genera scripts reproducibles.
- Mantiene la BD sincronizada con el código.

En esta fase:

1. Inicializamos Alembic.
 2. Configuramos `alembic.ini`.
 3. Ajustamos `alembic/env.py` para importar nuestros modelos.
 4. Creamos un archivo `migrate.py` que automatiza generación de migraciones + upgrade.
 5. Ejecutamos la migración completa.
-

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 5)

```
biblioteca_fastapi/  
├── alembic/                      (NUEVO - generado por Alembic)  
├── alembic.ini                  (NUEVO - generado por Alembic)  
├── migrate.py                   (NUEVO)  
├── models/  
│   └── libro.py  
├── database.py  
└── main.py
```

1 Inicializar Alembic

Desde la carpeta del proyecto:

```
alembic init alembic
```

Esto crea:

- Carpeta `alembic/`
 - Archivo `alembic.ini`
-

2 Configurar `alembic.ini`

Abre:

- `C:\Cursos\FastAPI\biblioteca_fastapi\alembic.ini`

Busca EXACTAMENTE esta línea:

```
sqlalchemy.url = driver://user:pass@localhost/dbname
```

Y sustitúyela COMPLETAMENTE por:

```
sqlalchemy.url = mysql+pymysql://root:admin@localhost:3306/biblioteca_db
```

Alembic usará esta URL para conectarse a la BD durante las migraciones.

3 Configurar `alembic/env.py`

Abrir:

- `C:\Cursos\FastAPI\biblioteca_fastapi\alembic\env.py`

A. Importar Base y modelos

Debes agregar al inicio (dentro del archivo):

```
from database import Base
from models.libro import Libro
```

B. Asignar target_metadata

Busca esta línea:

```
target_metadata = None
```

Y reemplázala **completamente** por:

```
target_metadata = Base.metadata
```

Esto permite que Alembic detecte los modelos (incluido `Libro`).



Archivo (NUEVO): `migrate.py`

Ruta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\migrate.py`

Este archivo automatiza:

- Cargar `.env`
- Verificar tabla `alembic_version`
- Crear migración autogenerada
- Aplicar migración (`upgrade head`)

Archivo completo listo para copiar y pegar:

```
# migrate.py
# Script para generar y aplicar migraciones con Alembic automáticamente.

import os
from dotenv import load_dotenv
from alembic.config import Config
from alembic import command
import pymysql

# Cargar variables del .env
```

```

load_dotenv()

DB_USER = os.getenv("DB_USER")
DB_PASSWORD = os.getenv("DB_PASSWORD")
DB_HOST = os.getenv("DB_HOST")
DB_PORT = os.getenv("DB_PORT")
DB_NAME = os.getenv("DB_NAME")

# Verificar si existe la base de datos y si existe la tabla alembic_version
def limpiar_alembic_version():
    try:
        conn = pymysql.connect(
            host=DB_HOST,
            user=DB_USER,
            password=DB_PASSWORD,
            database=DB_NAME
        )
        cursor = conn.cursor()
        cursor.execute("""
            SHOW TABLES LIKE 'alembic_version';
        """)
        existe = cursor.fetchone()

        if existe:
            print("✓ Tabla alembic_version encontrada. OK.")
        else:
            print("⚠ No existe tabla alembic_version. Continuando...")

        conn.close()
    except Exception as e:
        print("⚠ Error verificando alembic_version:", e)

# Ejecutar migración automática
def ejecutar_migracion():
    alembic_cfg = Config("alembic.ini")
    print("🔴 Generando revisión autogenerada...")
    command.revision(alembic_cfg, autogenerate=True, message="init libros")

    print("🔴 Aplicando migración (upgrade head)...")
    command.upgrade(alembic_cfg, "head")
    print("✓ Migración aplicada correctamente.")

if __name__ == "__main__":
    limpiar_alembic_version()
    ejecutar_migracion()

```

5 Ejecutar la migración

En consola:

```
python migrate.py
```

Deberías ver:

```
✖ Generando revisión autogenerada...
✖ Aplicando migración (upgrade head)...
✓ Migración aplicada correctamente.
```

Luego revisa MySQL Workbench → `base biblioteca_db`.

Debe existir la tabla:

- `libros`

Con columnas:

- `id`
- `titulo`
- `autor`
- `rating`

💡 Tips importantes

- No olvides importar tus modelos en `alembic/env.py`, si no → migración vacía.
- Si cambias modelos más adelante, solo ejecutas de nuevo: `python migrate.py`.
- Alembic controla versiones, así que no crees tablas manualmente.

¿Continuamos? Responde: **listo** para la Fase 6 (esquemas Pydantic y validaciones).

Dijiste:

listo vamos a la fase 6

ChatGPT dijo:

📖 FASE 6: ESQUEMAS Pydantic Y VALIDACIONES DE `Libro`

Por qué:

Hasta ahora solo tenemos el **modelo SQLAlchemy** (`Libro`), que representa la tabla en la base de datos. Pero para recibir y enviar datos desde la API necesitamos **esquemas Pydantic**, que se encargan de:

- Validar datos de entrada (request body).

- Definir qué se devuelve como salida (response model).
- Asegurar que la API nunca reciba ni entregue datos inválidos.
- Separar claramente “modelo de base de datos” vs “modelo de validación”.

En esta fase:

✓ Creamos `schemas.py`

✓ Definimos:

- LibroBase
- LibroCreate
- LibroUpdate
- LibroRead

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 6)

```

biblioteca_fastapi/
├── schemas.py          (NUEVO)
├── models/
│   └── libro.py
├── database.py
└── main.py

```



Archivo (NUEVO): `schemas.py`

Ruta exacta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\schemas.py`

Usando **Pydantic v2**, que es la versión actual y estable.
(Importante: no usar `@validator` de Pydantic v1).

Archivo COMPLETO listo para copiar y pegar:

```

# schemas.py
# Esquemas Pydantic para validación de datos y respuestas.

from pydantic import BaseModel, Field, field_validator
from typing import Optional

class LibroBase(BaseModel):
    """
    Esquema base con campos comunes a creación y actualización.
    Incluye validaciones de datos.

```

```

"""

titulo: str = Field(..., min_length=1, description="Título del libro")
autor: str = Field(..., min_length=1, description="Autor del libro")
rating: int = Field(
    ..., ge=1, le=5,
    description="Calificación del 1 al 5"
)

# Validación adicional opcional
@field_validator("titulo", "autor")
def no_vacios(cls, valor):
    if not valor.strip():
        raise ValueError("El campo no puede estar vacío")
    return valor

class LibroCreate(LibroBase):
    """
    Esquema para crear un libro.
    Hereda validaciones de LibroBase.
    """
    pass

class LibroUpdate(BaseModel):
    """
    Esquema para actualizar un libro.
    Permite que los campos sean opcionales.
    """

    titulo: Optional[str] = Field(None, min_length=1)
    autor: Optional[str] = Field(None, min_length=1)
    rating: Optional[int] = Field(None, ge=1, le=5)

    @field_validator("titulo", "autor")
    def no_vacios_opcional(cls, valor):
        if valor is not None and not valor.strip():
            raise ValueError("El campo no puede estar vacío")
        return valor

class LibroRead(BaseModel):
    """
    Esquema para las respuestas del API.
    Incluye el ID del libro.
    """

    id: int
    titulo: str
    autor: str
    rating: int

    # Permite devolver objetos ORM directamente
    model_config = {
        "from_attributes": True
    }

```

Explicación de los esquemas

LibroBase

- Esquema base.
- Usado como base para crear y actualizar.
- Incluye validaciones:
 - `min_length=1`
 - `rating` entre 1 y 5.
- `@field_validator` asegura que no estén vacíos ni hechos de espacios.

LibroCreate

- Usa todos los datos obligatorios.
- Simplemente hereda de `LibroBase`.

LibroUpdate

- Campos opcionales → permite actualizar solo algunos.
- Cada campo incluye validación si se envía.

LibroRead

- Respuesta que devolvemos al usuario.
- Incluye `id`.
- `model_config = {"from_attributes": True}`
Permite convertir un objeto SQLAlchemy → Pydantic fácilmente.

¿Continuamos? Responde: **listo** para la Fase 7 (capa de servicios / CRUD de libros).

Dijiste:

listo vamos a la fase 7

ChatGPT dijo:

FASE 7: CAPA DE SERVICIOS – FUNCIÓN `listar_libros`

Por qué:

Antes de exponer rutas en FastAPI, es importante tener una capa de **servicios** que encapsule la lógica de

negocio.

Esto permite:

- Mantener el router limpio.
- Reutilizar funciones (listar, crear, actualizar, eliminar).
- Separar responsabilidades (router ↔ lógica de negocio ↔ base de datos).

En esta fase:

- ✓ Creamos la carpeta `services`
- ✓ Creamos el archivo `libro_service.py`
- ✓ Implementamos la función **listar_libros**

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 7)

```

biblioteca_fastapi/
├── services/                (NUEVO)
│   └── libro_service.py    (NUEVO)
├── schemas.py
├── models/
│   └── libro.py
└── database.py
  
```



Archivo (NUEVO): `services/libro_service.py`

Ruta exacta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\services\libro_service.py`

Archivo completo listo para copiar y pegar:

```

# services/libro_service.py
# Capa de servicios encargada de la lógica relacionada con libros.

from sqlalchemy.orm import Session
from models.libro import Libro

def listar_libros(db: Session):
    """
    Devuelve una lista con todos los libros almacenados en la base de datos.

    Parámetros:
        db (Session): Sesión de base de datos proporcionada por FastAPI.
    """
  
```

```

Retorna:
    Lista de instancias del modelo Libro.
"""
return db.query(Libro).all()

```

Explicación de la capa de servicios

- **Por qué existe esta capa:**
Nos permite mantener el router ligero y ordenado. Toda la lógica de negocio vive aquí.
 - **Función `listar_libros`:**
 - Recibe la sesión `db` generada por la dependencia `get_db()`.
 - Ejecuta `db.query(Libro).all()` para obtener todos los registros.
 - Devuelve una lista de objetos `Libro` (ORM).
 - En fases posteriores agregaremos aquí:
 - crear libro
 - obtener por id
 - actualizar
 - eliminar
-

¿Continuamos? Responde: **listo** para la Fase 8 (router de libros – endpoint GET para listar).

Dijiste:

listo vamos a la fase 8

ChatGPT dijo:

FASE 8: ROUTER DE LIBROS – ENDPOINT `GET /api/libros` (LISTAR)

Por qué:

La capa de servicios ya puede listar libros, pero la API aún no expone ninguna ruta.

Ahora toca crear el **router** específico para libros, siguiendo buenas prácticas:

- Usar `APIRouter` para organizar endpoints.
- Agrupar todo bajo `/api/libros`.
- Separar rutas del archivo `main.py`.
- Utilizar `response_model` para devolver datos validados con Pydantic.

En esta fase:

- ✓ Creamos carpeta `api`
- ✓ Creamos archivo `libros.py`

- ✓ Agregamos endpoint GET /api/libros
- ✓ Modificamos main.py para incluir este router

📁 NUEVOS ARCHIVOS / MODIFICACIONES (Fase 8)

```

biblioteca_fastapi/
├── api/                                (NUEVO)
│   └── libros.py                      (NUEVO)
├── services/
│   └── libro_service.py
├── schemas.py
├── models/
│   └── libro.py
├── database.py
└── main.py                            (MODIFICADO)

```



Archivo (NUEVO): api/libros.py

Ruta exacta:

- C:\Cursos\FastAPI\biblioteca_fastapi\api\libros.py

Archivo **completo**, listo para copiar y pegar:

```

# api/libros.py
# Router encargado de exponer las rutas relacionadas con libros.

from fastapi import APIRouter, Depends
from typing import List
from sqlalchemy.orm import Session

from database import get_db
from services.libro_service import listar_libros
from schemas import LibroRead

# Crear router con prefijo y etiqueta
router = APIRouter(
    prefix="/api/libros",
    tags=["Libros"]
)

@router.get("/", response_model=List[LibroRead])
def obtener_libros(db: Session = Depends(get_db)):
    """
    Endpoint que devuelve la lista completa de libros.
    Utiliza la capa de servicios para acceder a la base de datos.
    """

```

```
libros = listar_libros(db)
return libros
```

Archivo (MODIFICADO): `main.py`

Ruta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\main.py`

A continuación el archivo **completo**, marcando las líneas nuevas con # [NUEVO]:

```
# main.py
# Punto de entrada de la aplicación FastAPI.

from fastapi import FastAPI

# [NUEVO] Importar router de libros
from api.libros import router as libros_router # [NUEVO]

app = FastAPI(
    title="API Biblioteca Personal",
    description="Backend REST con FastAPI para gestionar libros.",
    version="1.0.0"
)

@app.get("/")
def raiz():
    """
    Endpoint básico que confirma que la API está funcionando.
    """
    return {"mensaje": "API Biblioteca funcionando correctamente"}

# [NUEVO] Incluir router de libros
app.include_router(libros_router) # [NUEVO]
```

Explicación breve

✓ APIRouter

APIRouter permite:

- Organizar las rutas en módulos separados.
- Evitar que `main.py` crezca demasiado.
- Facilitar pruebas y mantenimiento.

✓ Response model (`List[LibroRead]`)

FastAPI:

- Convierte objetos SQLAlchemy → esquemas Pydantic.
 - Valida la salida antes de enviarla.
 - Documenta automáticamente en `/docs`.
-

Con esto ya puedes consultar:

GET `http://localhost:8000/api/libros`

Y verlo documentado en:

- `http://localhost:8000/docs`
-

¿Continuamos? Responde: **listo** para la Fase 9 (pruebas con Postman del listado de libros).

Dijiste:

listo vamos a la fase 9

ChatGPT dijo:

FASE 9: PRUEBAS CON POSTMAN – LISTAR LIBROS

Por qué:

Ya expusimos el primer endpoint real del proyecto (GET `/api/libros`).

Ahora debemos **probarlo cuidadosamente** antes de avanzar a las operaciones más complejas del CRUD. Probar temprano evita errores acumulados y asegura que toda la arquitectura (router → servicio → BD) funcione correctamente.

En esta fase:

- ✓ Insertamos datos de prueba en MySQL
- ✓ Probamos la API con Postman
- ✓ Revisamos la documentación automática `/docs`
- ✓ Confirmamos el flujo completo BD → servicio → router

No se modifica ningún archivo de código en esta fase.

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 9)


```
biblioteca_fastapi/  
├── api/  
│   └── libros.py  
├── services/  
│   └── libro_service.py  
├── schemas.py  
├── models/  
│   └── libro.py  
├── database.py  
└── main.py
```

(No hay archivos nuevos ni modificados)

1 Insertar 3 libros de ejemplo en MySQL

Ejecuta en **MySQL Workbench**:

```
INSERT INTO libros (titulo, autor, rating) VALUES  
( 'El Principito', 'Antoine de Saint-Exupéry', 5),  
( 'Cien años de soledad', 'Gabriel García Márquez', 5),  
( 'La Metamorfosis', 'Franz Kafka', 4);
```

Confirma que ahora la tabla tiene registros:

```
SELECT * FROM libros;
```

2 Levantar la API con Uvicorn

En la carpeta del proyecto:

```
uvicorn main:app --reload
```

Debe mostrar:

```
Uvicorn running on http://127.0.0.1:8000
```

3 Probar en Postman

✓ **Configuración:**

- **Método:** GET
- **URL:** `http://localhost:8000/api/libros`

✓ Respuesta esperada:

Una lista JSON como:

```
[
  {
    "id": 1,
    "titulo": "El Principito",
    "autor": "Antoine de Saint-Exupéry",
    "rating": 5
  },
  {
    "id": 2,
    "titulo": "Cien años de soledad",
    "autor": "Gabriel García Márquez",
    "rating": 5
  },
  {
    "id": 3,
    "titulo": "La Metamorfosis",
    "autor": "Franz Kafka",
    "rating": 4
  }
]
```

Si la tabla está vacía, el endpoint devolverá:

```
[]
```

4 Revisar la documentación automática

Abre:

- <http://localhost:8000/docs>

Ahí verás:

- Grupo **Libros**
- Endpoint **GET /api/libros**
- Modelo de respuesta `LibroRead`

Puedes probar el endpoint directamente desde ahí.

5 Si aparece algún error típico

Estos son los errores más comunes y cómo corregirlos:

✖ **Error: NameError: name 'LibroRead' is not defined**

Solución:

- Asegurarte de tener:
- `from schemas import LibroRead`

en `api/libros.py`.

✖ **Error: ModuleNotFoundError: No module named 'models'**

Solución:

- Verificar que la carpeta `models/` exista y tenga `__init__.py` (aunque Python 3.12 ya no lo requiere estrictamente).
- Confirmar que estás ejecutando desde la raíz del proyecto.

✖ **Error de conexión MySQL**

Solución:

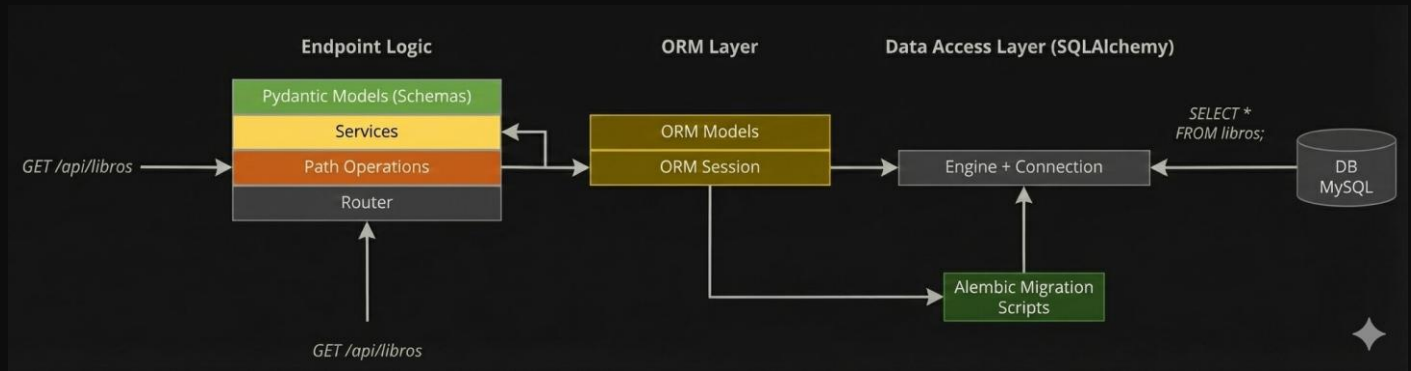
- Verificar `database.py` y `.env` (usuario, contraseña, puerto, base de datos).

Si aparece uno de estos errores, indícalo y lo corregimos marcando el archivo correspondiente como **(MODIFICADO)**.

Todo funcionando aquí significa que el flujo:

MySQL → SQLAlchemy → Servicio → Router → FastAPI → Postman

está 100% OK 🇵🇪



¿Continuamos? Responde: **listo** para la Fase 10 (capa de servicios: crear libro).

Dijiste:

listo vamos a la fase 10

ChatGPT dijo:



FASE 10: CAPA DE SERVICIOS – FUNCIÓN `crear_libro`

Por qué:

Ya podemos listar libros, pero para avanzar hacia un CRUD completo necesitamos la función que permita **crear nuevos registros** en la base de datos.

Esta función debe:

- Recibir un esquema `LibroCreate`.
- Convertirlo a un modelo SQLAlchemy.
- Guardarlo en la base de datos.
- Devolver el libro recién creado.

En esta fase:

- ✓ Ampliamos `libro_service.py`
- ✓ Mantenemos el archivo completo
- ✓ Marcamos SOLO lo nuevo con # `[NUEVO]`



NUEVOS ARCHIVOS / MODIFICACIONES (Fase 10)

```

biblioteca_fastapi/
├──
└── services/
  
```

```
| └─ libro_service.py      (MODIFICADO)
|
| └─ schemas.py
| └─ models/
|     └─ libro.py
| └─ database.py
| └─ main.py
```



Archivo (MODIFICADO): `services/libro_service.py`

Ruta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\services\libro_service.py`

Archivo **completo**, con todas las funciones implementadas hasta ahora
(y marcando únicamente las líneas nuevas con `# [NUEVO]`):

```
# services/libro_service.py
# Capa de servicios encargada de la lógica relacionada con libros.

from sqlalchemy.orm import Session
from models.libro import Libro
from schemas import LibroCreate # [NUEVO]

def listar_libros(db: Session):
    """
    Devuelve una lista con todos los libros almacenados en la base de datos.
    """
    return db.query(Libro).all()

def crear_libro(db: Session, datos: LibroCreate): # [NUEVO]
    """
    Crea un nuevo libro en la base de datos usando los datos validados
    del esquema LibroCreate.
    """
    # Crear instancia del modelo usando los datos recibidos
    nuevo_libro = Libro(
        titulo=datos.titulo,
        autor=datos.autor,
        rating=datos.rating
    ) # [NUEVO]

    # Guardar en la base de datos
    db.add(nuevo_libro) # [NUEVO]
    db.commit() # [NUEVO]
    db.refresh(nuevo_libro) # [NUEVO]

    return nuevo_libro # [NUEVO]
```



Explicación de `crear_libro`

- **Recibe:**
Un `LibroCreate`, ya validado por Pydantic.
 - **Crea:**
Una instancia del modelo SQLAlchemy (`Libro`).
 - **`db.add()`**
Marca el objeto para ser insertado.
 - **`db.commit()`**
Ejecuta la transacción en MySQL.
 - **`db.refresh()`**
Vuelve a cargar el objeto desde la BD, incluyendo su `id` autogenerado.
 - **Retorna:**
El libro recién insertado.
-

Con esto, la capa de servicios ya puede:

- Listar libros
- Crear libros

El siguiente paso es exponerlo en un endpoint `POST`.

¿Continuamos? Responde: **listo** para la Fase 11 (router: endpoint `POST /api/libros`).

Dijiste:

listo vamos a la fase 11

ChatGPT dijo:



FASE 11: ROUTER DE LIBROS – ENDPOINT `POST /api/libros` (CREAR)

Por qué:

La capa de servicios ya tiene la función `crear_libro`, pero el router aún no expone ninguna ruta para crear libros.

Esta fase conecta todo:



Frontend → Router → Servicio → MySQL

En esta fase:

- ✓ Ampliamos `api/libros.py`
- ✓ Agregamos el endpoint `POST /api/libros`
- ✓ Marcamos únicamente lo nuevo con `# [NUEVO]`
- ✓ Mantenemos el archivo completo

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 11)

```

biblioteca_fastapi/
├── api/
│   └── libros.py                (MODIFICADO)
├── services/
│   └── libro_service.py
├── schemas.py
├── models/
│   └── libro.py
├── database.py
└── main.py

```



Archivo (MODIFICADO): `api/libros.py`

Ruta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\api\libros.py`

Archivo **completo**, incluyendo el endpoint GET anterior
y marcando **solo las partes nuevas** con `# [NUEVO]`:

```

# api/libros.py
# Router encargado de exponer las rutas relacionadas con libros.

from fastapi import APIRouter, Depends, HTTPException # [NUEVO]
from typing import List
from sqlalchemy.orm import Session

from database import get_db
from services.libro_service import listar_libros, crear_libro # [NUEVO]
from schemas import LibroRead, LibroCreate # [NUEVO]

# Crear router con prefijo y etiqueta
router = APIRouter(
    prefix="/api/libros",
    tags=["Libros"]
)

# GET /api/libros
@router.get("/", response_model=List[LibroRead])

```

```
def obtener_libros(db: Session = Depends(get_db)):
    """
    Devuelve la lista completa de libros usando la capa de servicios.
    """
    libros = listar_libros(db)
    return libros

# POST /api/libros ----- [NUEVO]
@router.post("/", response_model=LibroRead, status_code=201) # [NUEVO]
def crear_libro_endpoint(datos: LibroCreate, db: Session = Depends(get_db)): # [NUEVO]
    """
    Crea un nuevo libro en la base de datos.
    Recibe un cuerpo JSON validado con LibroCreate. # [NUEVO]
    """ # [NUEVO]
    nuevo_libro = crear_libro(db, datos) # [NUEVO]
    return nuevo_libro # [NUEVO]
```



Explicación del endpoint POST

✓ **response_model=LibroRead**

La salida se valida con Pydantic y se documenta automáticamente en /docs.

✓ **status_code=201**

Código HTTP estándar para **recurso creado**.

✓ **datos: LibroCreate**

FastAPI se encarga de:

- Leer el body JSON.
- Validarlo con Pydantic.
- Convertirlo a un objeto `LibroCreate`.

✓ **Uso de la capa de servicios**

`crear_libro(db, datos)` mantiene la lógica de creación centralizada.

Con esto, ya puedes crear libros con:

POST `http://localhost:8000/api/libros`

Body JSON:


```
{
  "titulo": "1984",
  "autor": "George Orwell",
  "rating": 5
}
```

¿Continuamos? Responde: **listo** para la Fase 12 (pruebas con Postman – crear libro).

Dijiste:

listo vamos a la fase 12

ChatGPT dijo:

FASE 12: PRUEBAS CON POSTMAN – CREAR LIBRO

Por qué:

Ya implementamos el endpoint `POST /api/libros`, pero antes de avanzar debemos **probarlo minuciosamente**.

Aquí confirmamos que:

- La validación Pydantic funciona.
- La creación se realiza correctamente en MySQL.
- El ID autoincremental se asigna.
- La API devuelve el objeto recién creado.

En esta fase **no hubo cambios en el código**, solo pruebas.

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 12)

```
biblioteca_fastapi/
├── api/
│   └── libros.py
├── services/
│   └── libro_service.py
├── schemas.py
├── models/
│   └── libro.py
├── database.py
└── main.py
```

(No hay archivos nuevos ni modificados)

Levantar la API

En la raíz del proyecto:

```
uvicorn main:app --reload
```

2 Probar creación en Postman

✓ Configuración:

- **Método:** POST
- **URL:** `http://localhost:8000/api/libros`
- **Body:** selecciona *raw* → JSON

Ejemplo de JSON válido:

```
{
  "titulo": "1984",
  "autor": "George Orwell",
  "rating": 5
}
```

✓ Respuesta esperada:

```
{
  "id": 4,
  "titulo": "1984",
  "autor": "George Orwell",
  "rating": 5
}
```

El `id` variará según tus datos existentes.

3 Validar con `GET /api/libros`

Después del POST, ejecuta:

```
GET http://localhost:8000/api/libros
```

Deberías ver el libro recién agregado.

4 Revisar en MySQL Workbench

Consulta directamente:

```
SELECT * FROM libros;
```

Debe incluir el registro nuevo.

5 Verificar el endpoint en la documentación automática

Visita:

- <http://localhost:8000/docs>

Busca el grupo **Libros**, y dentro:

- POST /api/libros
 - Prueba desde ahí también si lo deseas.
-

6 Pruebas de validación (muy importante)

✗ Caso rating fuera de rango:

```
{
  "titulo": "Libro inválido",
  "autor": "Autor",
  "rating": 6
}
```

Respuesta esperada (400):

```
{
  "detail": [
    {
      "type": "greater_than_or_equal",
      "loc": ["body", "rating"],
      "msg": "Input should be less than or equal to 5"
    }
  ]
}
```

✗ Caso título vacío:

```
{
  "titulo": "",
  "autor": "Autor",
}
```

```
"rating": 3
}
```

Respuesta:

```
{
  "detail": [
    {
      "msg": "El campo no puede estar vacío",
      "loc": ["body", "titulo"],
      "type": "value_error"
    }
  ]
}
```

Esto confirma que las validaciones Pydantic están funcionando correctamente.

Todo funcionando aquí significa que ya podemos continuar con la lectura individual (GET /api/libros/{id}).

¿Continuamos? Responde: **listo** para la Fase 13 (capa de servicios: obtener libro por id).

Dijiste:

listo vamos a la fase 13

ChatGPT dijo:



FASE 13: CAPA DE SERVICIOS – FUNCIÓN `obtener_libro_por_id`

Por qué:

Para completar el CRUD necesitamos la capacidad de **consultar un libro específico** por su ID.

Antes de exponer el endpoint, primero implementamos esta función en la **capa de servicios**, manteniendo una arquitectura limpia:

- Router → Servicio → Base de datos
- El router no debe contener lógica de negocio.
- El servicio se encarga de obtener el libro y manejar el caso *no encontrado* (devolviendo `None` para que el router decida qué hacer).

En esta fase:

- ✓ Actualizamos `libro_service.py`
- ✓ Agregamos la función `obtener_libro_por_id`

- ✓ Mantenemos el archivo completo
- ✓ Marcamos únicamente lo nuevo con # [NUEVO]

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 13)

```
biblioteca_fastapi/  
├── services/  
│   └── libro_service.py    (MODIFICADO)  
├── api/  
│   └── libros.py  
├── schemas.py  
├── models/  
│   └── libro.py  
├── database.py  
└── main.py
```



Archivo (MODIFICADO): `services/libro_service.py`

Ruta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\services\libro_service.py`

Archivo **completo**, con todas las funciones hasta ahora
y marcando lo nuevo con # [NUEVO]:

```
# services/libro_service.py  
# Capa de servicios encargada de la lógica relacionada con libros.  
  
from sqlalchemy.orm import Session  
from models.libro import Libro  
from schemas import LibroCreate  
  
def listar_libros(db: Session):  
    """  
    Devuelve una lista con todos los libros almacenados en la base de datos.  
    """  
    return db.query(Libro).all()  
  
def crear_libro(db: Session, datos: LibroCreate):  
    """  
    Crea un nuevo libro en la base de datos usando los datos validados  
    del esquema LibroCreate.  
    """  
    nuevo_libro = Libro(  

```

```

        titulo=datos.titulo,
        autor=datos.autor,
        rating=datos.rating
    )
    db.add(nuevo_libro)
    db.commit()
    db.refresh(nuevo_libro)
    return nuevo_libro

def obtener_libro_por_id(db: Session, id: int):    # [NUEVO]
    """
    Obtiene un libro por su ID.
    Retorna el libro si existe; de lo contrario retorna None.
    """                                          # [NUEVO]
    return db.query(Libro).filter(Libro.id == id).first()    # [NUEVO]

```



Explicación

✓ `db.query(...).filter(...).first()`

- Busca el primer libro cuyo ID coincida.
- Si no existe, devuelve `None`.
- Esto permite al router decidir si devuelve el libro o un error 404.

✓ Separación de responsabilidades

- La capa de servicios **no** lanza excepciones HTTP.
- El router será quien decida qué hacer si recibe `None`.

La capa de servicios ahora puede:

- Listar libros
- Crear libros
- Obtener libro por ID

Listo para el siguiente paso: exponerlo en el router.

¿Continuamos? Responde: **listo** para la Fase 14 (router: endpoint GET `/api/libros/{id}`).

Dijiste:

listo vamos a la fase 14

ChatGPT dijo:

FASE 14: ROUTER DE LIBROS – ENDPOINT `GET /api/libros/{id}`

Por qué:

La capa de servicios ya puede buscar un libro por su ID, pero aún no hemos expuesto esa funcionalidad como un endpoint REST.

Ahora agregaremos:

- `GET /api/libros/{id}`
- Validación del parámetro `id`
- Manejo del caso *no encontrado* con `HTTPException(404)`
- Uso de `LibroRead` como modelo de respuesta

En esta fase:

- ✓ Modificamos `api/libros.py`
- ✓ Manteniendo el archivo completo
- ✓ Marcando solo lo nuevo con # [NUEVO]

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 14)

```

biblioteca_fastapi/
├── api/
│   └── libros.py                (MODIFICADO)
├── services/
│   └── libro_service.py
├── schemas.py
├── models/
│   └── libro.py
├── database.py
└── main.py
  
```



Archivo (MODIFICADO): `api/libros.py`

Ruta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\api\libros.py`

Archivo **completo**, con todos los endpoints hasta ahora, marcando lo nuevo con # [NUEVO]:

```

# api/libros.py
# Router encargado de exponer las rutas relacionadas con libros.
  
```

```

from fastapi import APIRouter, Depends, HTTPException, Path # [NUEVO]
from typing import List
from sqlalchemy.orm import Session

from database import get_db
from services.libro_service import (
    listar_libros,
    crear_libro,
    obtener_libro_por_id # [NUEVO]
)
from schemas import LibroRead, LibroCreate

# Crear router con prefijo y etiqueta
router = APIRouter(
    prefix="/api/libros",
    tags=["Libros"]
)

# GET /api/libros (listar)
@router.get("/", response_model=List[LibroRead])
def obtener_libros(db: Session = Depends(get_db)):
    """
    Devuelve la lista completa de libros usando la capa de servicios.
    """
    libros = listar_libros(db)
    return libros

# POST /api/libros (crear)
@router.post("/", response_model=LibroRead, status_code=201)
def crear_libro_endpoint(datos: LibroCreate, db: Session = Depends(get_db)):
    """
    Crea un nuevo libro en la base de datos.
    """
    nuevo_libro = crear_libro(db, datos)
    return nuevo_libro

# GET /api/libros/{id} ----- [NUEVO]
@router.get("/{id}", response_model=LibroRead) # [NUEVO]
def obtener_libro_endpoint(
    id: int = Path(..., gt=0, description="ID del libro a consultar"), # [NUEVO]
    db: Session = Depends(get_db) # [NUEVO]
): # [NUEVO]
    """
    Devuelve un libro específico según su ID.
    Lanza error 404 si no existe.
    """
    libro = obtener_libro_por_id(db, id)

    if not libro:
        raise HTTPException(
            status_code=404,
            detail=f"Libro con id {id} no encontrado"
        )

```



```
return libro
```

[NUEVO]



Explicación del endpoint

✓ `Path(..., gt=0)`

Valida que el ID:

- Sea obligatorio
- Sea un entero
- Sea mayor que 0

✓ Manejo del caso no encontrado

Si la capa de servicios devuelve `None`, el router lanza:

```
HTTPException(404, detail="Libro con id X no encontrado")
```

✓ `response_model=LibroRead`

Garantiza que la salida siempre tenga:

- id
- titulo
- autor
- rating

Con esto ya puedes probar:

```
GET http://localhost:8000/api/libros/1
```

¿Continuamos? Responde: **listo** para la Fase 15 (pruebas con Postman – obtener libro por id).

Dijiste:

listo vamos a la fase 15

ChatGPT dijo:



FASE 15: PRUEBAS CON POSTMAN – OBTENER LIBRO POR ID

Por qué:

Ya implementamos el endpoint `GET /api/libros/{id}`, pero antes de avanzar debemos **validar cuidadosamente** su funcionamiento.

Queremos confirmar que:

- La API devuelve correctamente un libro existente.
- Maneja adecuadamente el error 404 cuando el ID no existe.
- La documentación automática `/docs` describe bien el endpoint.

En esta fase **no se modifica código**, solamente se realizan pruebas.

📁 NUEVOS ARCHIVOS / MODIFICACIONES (Fase 15)

```
biblioteca_fastapi/  
├── api/  
│   └── libros.py  
├── services/  
│   └── libro_service.py  
├── schemas.py  
├── models/  
│   └── libro.py  
├── database.py  
└── main.py
```

(No hay archivos nuevos ni modificados)

1 Probar libro existente en Postman

Suponiendo que ya ingresaste datos en fases anteriores, prueba:

- **Método:** GET
- **URL:** `http://localhost:8000/api/libros/1`

Respuesta esperada:

```
{  
  "id": 1,  
  "titulo": "El Principito",  
  "autor": "Antoine de Saint-Exupéry",  
  "rating": 5  
}
```

Si tienes otros datos, la respuesta variará.

2 Probar libro que NO existe

Prueba:

- GET `http://localhost:8000/api/libros/9999`

Respuesta esperada (404):

```
{  
  "detail": "Libro con id 9999 no encontrado"  
}
```

Esto confirma que:

- El servicio devuelve `None` cuando no existe.
 - El router transforma ese `None` en un `HTTPException(404)`.
-

3 Revisar la documentación automática

Abre:

- `http://localhost:8000/docs`

Ahí verás:

- Grupo **Libros**
- Endpoint **GET /api/libros/{id}**
- Parámetro `id` validado (`integer > 0`)
- Modelo de respuesta `LibroRead`

Puedes probar el endpoint directamente desde Swagger UI.

4 Pruebas adicionales útiles

✓ ID inválido (negativo)

GET `/api/libros/-5`

FastAPI mostrará error de validación:

`greater_than`

✓ ID no numérico

GET /api/libros/abc

FastAPI devuelve error 422 indicando que `abc` no es un valor entero.

Todo correcto aquí significa que la lectura individual está funcionando perfectamente.

¿Continuamos? Responde: **listo** para la Fase 16 (capa de servicios: actualizar libro).

Dijiste:

listo vamos a la fase 16

ChatGPT dijo:



FASE 16: CAPA DE SERVICIOS – FUNCIÓN `actualizar_libro`

Por qué:

Ya podemos crear, listar y obtener libros por ID.

Ahora necesitamos la lógica de negocio para **actualizar** un libro existente.

Esta función debe:

- Buscar el libro por ID.
- Si no existe → retornar `None` (el router decidirá si lanzar 404).
- Actualizar únicamente los campos enviados (parcial o completamente).
- Guardar los cambios en la base de datos.

En esta fase:

- ✓ Modificamos `libro_service.py`
 - ✓ Mantenemos el archivo completo
 - ✓ Agregamos la función `actualizar_libro`
 - ✓ Marcamos solo las partes nuevas con `# [NUEVO]`
-



NUEVOS ARCHIVOS / MODIFICACIONES (Fase 16)

```

biblioteca_fastapi/
├── services/

```

```
|   └─ libro_service.py      (MODIFICADO)
|
|   └─ api/
|       └─ libros.py
|   └─ schemas.py
|   └─ models/
|       └─ libro.py
|   └─ database.py
|   └─ main.py
```



Archivo (MODIFICADO): `services/libro_service.py`

Ruta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\services\libro_service.py`

Archivo **completo**, con todas las funciones hasta ahora
y marcando únicamente lo nuevo con `# [NUEVO]`:

```
# services/libro_service.py
# Capa de servicios encargada de la lógica relacionada con libros.

from sqlalchemy.orm import Session
from models.libro import Libro
from schemas import LibroCreate, LibroUpdate # [NUEVO]

def listar_libros(db: Session):
    """
    Devuelve una lista con todos los libros almacenados en la base de datos.
    """
    return db.query(Libro).all()

def crear_libro(db: Session, datos: LibroCreate):
    """
    Crea un nuevo libro en la base de datos usando los datos validados
    del esquema LibroCreate.
    """
    nuevo_libro = Libro(
        titulo=datos.titulo,
        autor=datos.autor,
        rating=datos.rating
    )
    db.add(nuevo_libro)
    db.commit()
    db.refresh(nuevo_libro)
    return nuevo_libro

def obtener_libro_por_id(db: Session, id: int):
    """
    Obtiene un libro por su ID.
```

```

Retorna el libro si existe; de lo contrario retorna None.
"""
return db.query(Libro).filter(Libro.id == id).first()

def actualizar_libro(db: Session, id: int, datos: LibroUpdate): # [NUEVO]
    """
    Actualiza un libro existente.
    Solo se modifican los campos que el usuario haya enviado. # [NUEVO]
    Retorna el libro actualizado o None si no existe. # [NUEVO]
    """ # [NUEVO]

    libro = obtener_libro_por_id(db, id) # [NUEVO]
    if not libro: # [NUEVO]
        return None # [NUEVO]

    # Actualización parcial: solo los campos enviados # [NUEVO]
    if datos.titulo is not None: # [NUEVO]
        libro.titulo = datos.titulo # [NUEVO]

    if datos.autor is not None: # [NUEVO]
        libro.autor = datos.autor # [NUEVO]

    if datos.rating is not None: # [NUEVO]
        libro.rating = datos.rating # [NUEVO]

    db.commit() # [NUEVO]
    db.refresh(libro) # [NUEVO]

    return libro # [NUEVO]

```



Explicación de `actualizar_libro`

✓ Actualización parcial (PATCH-like)

Aunque usamos `PUT`, este método permite actualizar:

- Solo un campo (por ejemplo, solo `rating`)
- O varios campos
- O todos los campos

Gracias a que `LibroUpdate` tiene campos opcionales.

✓ Flujo

1. Buscar libro por ID.
2. Si no existe → retorna `None`.
3. Si existe → verificar qué campos envió el usuario.
4. Actualizar únicamente esos campos.

5. Guardar cambios (`commit`).
6. Recargar datos (`refresh`).
7. Retornar el libro actualizado.

✓ Router se encargará del 404

Esta capa **no debe lanzar exceptions HTTP**, solo regresar `None`.

La capa de servicios ya soporta:

- Listar
 - Crear
 - Obtener por id
 - Actualizar
-

¿Continuamos? Responde: **listo** para la Fase 17 (router: endpoint PUT `/api/libros/{id}`).

Dijiste:

listo vamos a la fase 17

ChatGPT dijo:

FASE 17: ROUTER DE LIBROS – ENDPOINT PUT `/api/libros/{id}`

Por qué:

Ya tenemos lista la función `actualizar_libro` en la capa de servicios. Ahora necesitamos exponer esa funcionalidad a través de un endpoint REST:

- Método: **PUT**
- Ruta: **`/api/libros/{id}`**
- Entrada: **LibroUpdate**
- Salida: **LibroRead**
- Respuestas:
 - 200 → libro actualizado
 - 404 → libro no encontrado

En esta fase:

- ✓ Modificamos `api/libros.py`
- ✓ Mantenemos el archivo completo
- ✓ Marcamos solo las partes nuevas con `# [NUEVO]`

📁 NUEVOS ARCHIVOS / MODIFICACIONES (Fase 17)

```

biblioteca_fastapi/
├── api/
│   └── libros.py      (MODIFICADO)
├── services/
│   └── libro_service.py
├── schemas.py
├── models/
│   └── libro.py
├── database.py
└── main.py

```



Archivo (MODIFICADO): api/libros.py

Ruta:

- C:\Cursos\FastAPI\biblioteca_fastapi\api\libros.py

Archivo **completo**, marcando únicamente lo nuevo:

```

# api/libros.py
# Router encargado de exponer las rutas relacionadas con libros.

from fastapi import APIRouter, Depends, HTTPException, Path
from typing import List
from sqlalchemy.orm import Session

from database import get_db
from services.libro_service import (
    listar_libros,
    crear_libro,
    obtener_libro_por_id,
    actualizar_libro # [NUEVO]
)
from schemas import LibroRead, LibroCreate, LibroUpdate # [NUEVO]

# Crear router con prefijo y etiqueta
router = APIRouter(
    prefix="/api/libros",
    tags=["Libros"]
)

# GET /api/libros (listar)
@router.get("/", response_model=List[LibroRead])
def obtener_libros(db: Session = Depends(get_db)):
    """
    Devuelve la lista completa de libros usando la capa de servicios.

```



```

"""
libros = listar_libros(db)
return libros

# POST /api/libros (crear)
@router.post("/", response_model=LibroRead, status_code=201)
def crear_libro_endpoint(datos: LibroCreate, db: Session = Depends(get_db)):
    """
    Crea un nuevo libro en la base de datos.
    """
    nuevo_libro = crear_libro(db, datos)
    return nuevo_libro

# GET /api/libros/{id}
@router.get("/{id}", response_model=LibroRead)
def obtener_libro_endpoint(
    id: int = Path(..., gt=0, description="ID del libro a consultar"),
    db: Session = Depends(get_db)
):
    """
    Devuelve un libro específico según su ID.
    Lanza error 404 si no existe.
    """
    libro = obtener_libro_por_id(db, id)

    if not libro:
        raise HTTPException(
            status_code=404,
            detail=f"Libro con id {id} no encontrado"
        )
    return libro

# PUT /api/libros/{id} ----- [NUEVO]
@router.put(
   ("/{id}",
    response_model=LibroRead,
    status_code=200
) # [NUEVO]
def actualizar_libro_endpoint(
    id: int = Path(..., gt=0, description="ID del libro a actualizar"), # [NUEVO]
    datos: LibroUpdate = None, # [NUEVO]
    db: Session = Depends(get_db) # [NUEVO]
): # [NUEVO]
    """
    Actualiza un libro existente. # [NUEVO]
    Lanza error 404 si el libro no existe. # [NUEVO]
    """ # [NUEVO]

    libro_actualizado = actualizar_libro(db, id, datos) # [NUEVO]

    if not libro_actualizado: # [NUEVO]
        raise HTTPException( # [NUEVO]
            status_code=404,
            detail=f"Libro con id {id} no encontrado"

```

```
)  
  
return libro_actualizado # [NUEVO]
```

Explicación del endpoint

✓ Entrada: `LibroUpdate`

Permite actualizar uno o varios campos:

```
{  
    "titulo": "Nuevo título",  
    "rating": 4  
}
```

✓ Validación del ID

`Path(..., gt=0)` garantiza que el ID sea válido.

✓ Manejo del error 404

Si la función en servicios regresa `None`, el router responde:

```
HTTPException(404, "Libro con id X no encontrado")
```

✓ Respuesta final

Devuelve el libro ya actualizado, como `LibroRead`.

La API ya soporta:

- Listar
- Crear
- Obtener por ID
- **Actualizar**

Vamos perfectos.

¿Continuamos? Responde: **listo** para la Fase 18 (pruebas con Postman – actualizar libro).

Dijiste:

listo vamos a la fase 18

ChatGPT dijo:

FASE 18: PRUEBAS CON POSTMAN – ACTUALIZAR LIBRO

Por qué:

Ahora que ya implementamos el endpoint `PUT /api/libros/{id}`, debemos asegurarnos de que la actualización funcione correctamente:

- Que permita actualizar uno o varios campos.
- Que responda con el libro actualizado.
- Que arroje 404 si el libro no existe.
- Que valide correctamente los datos (ej: rating debe estar entre 1 y 5).

En esta fase **no se modifica código**, solamente se realizan pruebas.

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 18)

```
biblioteca_fastapi/  
├── api/  
│   └── libros.py  
├── services/  
│   └── libro_service.py  
├── schemas.py  
├── models/  
│   └── libro.py  
├── database.py  
└── main.py
```

(No hay archivos nuevos ni modificados)

Levanta la API

En tu consola:

```
uvicorn main:app --reload
```

Se abre en:

- <http://localhost:8000/api/libros>
- <http://localhost:8000/docs>

2 Prueba PUT /api/libros/{id} en Postman

✓ Ejemplo: Actualizar todos los campos

Método: PUT

URL: `http://localhost:8000/api/libros/1`

Body (JSON):

```
{
  "titulo": "El Hobbit",
  "autor": "J. R. R. Tolkien",
  "rating": 5
}
```

Respuesta esperada:

```
{
  "id": 1,
  "titulo": "El Hobbit",
  "autor": "J. R. R. Tolkien",
  "rating": 5
}
```

3 Actualización parcial (solo un campo)

Gracias al esquema `LibroUpdate`, puedes enviar solo uno:

✓ Ejemplo: modificar solo el rating

```
{
  "rating": 4
}
```

Respuesta:

```
{
  "id": 1,
  "titulo": "El Hobbit",
  "autor": "J. R. R. Tolkien",
  "rating": 4
}
```

4 Caso: libro inexistente

Prueba:

PUT <http://localhost:8000/api/libros/9999>

Con cualquier JSON válido.

Respuesta esperada:

```
{
  "detail": "Libro con id 9999 no encontrado"
}
```

Código de estado: 404.

5 Validación de datos (rating inválido)

Ejemplo incorrecto:

```
{
  "rating": 10
}
```

Respuesta:

```
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": ["body", "rating"],
      "msg": "Input should be less than or equal to 5",
      ...
    }
  ]
}
```

Esto demuestra que **Pydantic** está validando correctamente los valores.

6 Revisar en Swagger UI (/docs)

En la documentación debes ver:

- Endpoint PUT `/api/libros/{id}`
- Descripción del parámetro `id`
- Modelo de entrada `LibroUpdate`
- Modelo de salida `LibroRead`

Swagger también permite probar la actualización sin Postman.

Cuando todo esto funciona, la operación de actualización está lista.

¿Continuamos? Responde: **listo** para la Fase 19 (capa de servicios: eliminar libro).

Dijiste:

listo vamos a la fase 19

ChatGPT dijo:

FASE 19: CAPA DE SERVICIOS – FUNCIÓN `eliminar_libro`

Por qué:

Ya podemos crear, listar, obtener y actualizar libros.

El siguiente paso para completar el CRUD es implementar la función que permitirá **eliminar un libro de la base de datos**.

Esta función debe:

- Buscar el libro por ID.
- Si no existe → retornar `None` (el router se encargará del 404).
- Si existe → eliminarlo de la base de datos.
- Confirmar la eliminación aplicando `commit()`.

En esta fase:

- ✓ Modificamos `services/libro_service.py`
 - ✓ Mantenemos el archivo completo
 - ✓ Agregamos la función `eliminar_libro`
 - ✓ Marcamos únicamente lo nuevo con # [NUEVO]
-

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 19)

```
biblioteca_fastapi/
├── services/
│   └── libro_service.py    (MODIFICADO)
└── api/
```

```
| └─ libros.py
| └─ schemas.py
| └─ models/
|     └─ libro.py
| └─ database.py
| └─ main.py
```



Archivo (MODIFICADO): `services/libro_service.py`

Ruta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\services\libro_service.py`

Archivo **completo**, marcando solo lo nuevo:

```
# services/libro_service.py
# Capa de servicios encargada de la lógica relacionada con libros.

from sqlalchemy.orm import Session
from models.libro import Libro
from schemas import LibroCreate, LibroUpdate

def listar_libros(db: Session):
    """
    Devuelve una lista con todos los libros almacenados en la base de datos.
    """
    return db.query(Libro).all()

def crear_libro(db: Session, datos: LibroCreate):
    """
    Crea un nuevo libro en la base de datos usando los datos validados
    del esquema LibroCreate.
    """
    nuevo_libro = Libro(
        titulo=datos.titulo,
        autor=datos.autor,
        rating=datos.rating
    )
    db.add(nuevo_libro)
    db.commit()
    db.refresh(nuevo_libro)
    return nuevo_libro

def obtener_libro_por_id(db: Session, id: int):
    """
    Obtiene un libro por su ID.
    Retorna el libro si existe; de lo contrario retorna None.
    """
    return db.query(Libro).filter(Libro.id == id).first()
```

```

def actualizar_libro(db: Session, id: int, datos: LibroUpdate):
    """
    Actualiza un libro existente.
    Solo se modifican los campos enviados.
    Retorna el libro actualizado o None si no existe.
    """
    libro = obtener_libro_por_id(db, id)
    if not libro:
        return None

    if datos.titulo is not None:
        libro.titulo = datos.titulo

    if datos.autor is not None:
        libro.autor = datos.autor

    if datos.rating is not None:
        libro.rating = datos.rating

    db.commit()
    db.refresh(libro)

    return libro

def eliminar_libro(db: Session, id: int): # [NUEVO]
    """
    Elimina un libro de la base de datos si existe. # [NUEVO]
    Retorna True si se eliminó, o None si no existe. # [NUEVO]
    """ # [NUEVO]

    libro = obtener_libro_por_id(db, id) # [NUEVO]
    if not libro: # [NUEVO]
        return None # [NUEVO]

    db.delete(libro) # [NUEVO]
    db.commit() # [NUEVO]

    return True # [NUEVO]

```



Explicación de `eliminar_libro`

✓ Flujo de eliminación

1. Buscar el libro → si no existe → None.
2. Si existe:
 - o `db.delete(libro)`
 - o `db.commit()`
3. Regresar `True` indicando éxito.

✓ No lanzar errores HTTP aquí

La capa de servicios **no** lanza excepciones HTTP.
Eso es responsabilidad del **router**, que decidirá cuándo devolver 404.

La capa de servicios ahora incluye:

- Listar
- Crear
- Obtener por id
- Actualizar
- **Eliminar ✓**

CRUD completo en backend.

¿Continuamos? Responde: **listo** para la Fase 20 (router: endpoint DELETE /api/libros/{id}).

Dijiste:

listo vamos a la fase 20

ChatGPT dijo:

FASE 20: ROUTER DE LIBROS – ENDPOINT DELETE /api/libros/{id}

Por qué:

Ya implementamos en la capa de servicios la función `eliminar_libro`.
Ahora debemos exponerla mediante un endpoint REST para completar el CRUD.

El endpoint debe:

- Recibir un **ID** vía `Path`.
- Llamar a `eliminar_libro`.
- Responder:
 - **204 (No Content)** → si se eliminó correctamente.
 - **404** → si el libro no existe.

En esta fase:

- ✓ Se modifica `api/libros.py`
 - ✓ Se muestra **el archivo completo**
 - ✓ Se marcan solo las líneas nuevas con # [NUEVO]
-

📁 NUEVOS ARCHIVOS / MODIFICACIONES (Fase 20)

```

biblioteca_fastapi/
├── api/
│   └── libros.py      (MODIFICADO)
├── services/
│   └── libro_service.py
├── schemas.py
├── models/
│   └── libro.py
├── database.py
└── main.py

```



Archivo (MODIFICADO): api/libros.py

Ruta:

- C:\Cursos\FastAPI\biblioteca_fastapi\api\libros.py

Archivo **completo**, con el nuevo endpoint DELETE marcado:

```

# api/libros.py
# Router encargado de exponer las rutas relacionadas con libros.

from fastapi import APIRouter, Depends, HTTPException, Path
from typing import List
from sqlalchemy.orm import Session

from database import get_db
from services.libro_service import (
    listar_libros,
    crear_libro,
    obtener_libro_por_id,
    actualizar_libro,
    eliminar_libro,      # [NUEVO]
)
from schemas import LibroRead, LibroCreate, LibroUpdate

# Crear router con prefijo y etiqueta
router = APIRouter(
    prefix="/api/libros",
    tags=["Libros"]
)

# GET /api/libros (listar)
@router.get("/", response_model=List[LibroRead])
def obtener_libros(db: Session = Depends(get_db)):
    """
    Devuelve la lista completa de libros usando la capa de servicios.
    """

```

```

    return listar_libros(db)

# POST /api/libros (crear)
@router.post("/", response_model=LibroRead, status_code=201)
def crear_libro_endpoint(datos: LibroCreate, db: Session = Depends(get_db)):
    """
    Crea un nuevo libro en la base de datos.
    """
    return crear_libro(db, datos)

# GET /api/libros/{id}
@router.get("/{id}", response_model=LibroRead)
def obtener_libro_endpoint(
    id: int = Path(..., gt=0, description="ID del libro a consultar"),
    db: Session = Depends(get_db)
):
    """
    Devuelve un libro específico según su ID.
    """
    libro = obtener_libro_por_id(db, id)
    if not libro:
        raise HTTPException(
            status_code=404,
            detail=f"Libro con id {id} no encontrado"
        )
    return libro

# PUT /api/libros/{id}
@router.put(
   ("/{id}",
    response_model=LibroRead,
    status_code=200
)
def actualizar_libro_endpoint(
    id: int = Path(..., gt=0, description="ID del libro a actualizar"),
    datos: LibroUpdate = None,
    db: Session = Depends(get_db)
):
    """
    Actualiza un libro existente.
    """
    libro_actualizado = actualizar_libro(db, id, datos)

    if not libro_actualizado:
        raise HTTPException(
            status_code=404,
            detail=f"Libro con id {id} no encontrado"
        )
    return libro_actualizado

# DELETE /api/libros/{id} ----- [NUEVO]
@router.delete(
   ("/{id}",

```

```

        status_code=204
    ) # [NUEVO]
def eliminar_libro_endpoint(
    id: int = Path(..., gt=0, description="ID del libro a eliminar"), # [NUEVO]
    db: Session = Depends(get_db) # [NUEVO]
): # [NUEVO]
    """
    Elimina un libro por su ID. # [NUEVO]
    Devuelve 204 si se eliminó, o 404 si no existe. # [NUEVO]
    """ # [NUEVO]

    resultado = eliminar_libro(db, id) # [NUEVO]

    if not resultado: # [NUEVO]
        raise HTTPException( # [NUEVO]
            status_code=404, # [NUEVO]
            detail=f"Libro con id {id} no encontrado"
        )
    return None # [NUEVO]

```



Explicación del endpoint

✓ 204 No Content

Se usa cuando la eliminación fue exitosa y **no hay contenido que devolver**.

✓ Manejo de error 404

Si la capa de servicios devuelve `None`, significa que el libro **no existe**, y el router responde:

```
HTTPException(404, "Libro con id X no encontrado")
```

✓ Limpieza en la respuesta

`return None` es la forma correcta para que FastAPI envíe un 204 sin contenido.

La API ahora soporta el CRUD completo:

- Crear
 - Listar
 - Obtener por id
 - Actualizar
 - **Eliminar ✓**
-

¿Continuamos? Responde: **listo** para la Fase 21 (pruebas con Postman – eliminar libro).

Dijiste:

listo vamos a la fase 21

ChatGPT dijo:

FASE 21: PRUEBAS CON POSTMAN – ELIMINAR LIBRO

Por qué:

Ya implementamos el endpoint `DELETE /api/libros/{id}`, pero necesitamos validar que funcione correctamente en todos los escenarios:

- Elimina libros existentes.
- Devuelve código **204 (No Content)** cuando la eliminación es exitosa.
- Devuelve **404** cuando el libro no existe.
- Refleja los cambios en consultas posteriores (`GET`).

En esta fase **no se modifica código**, solo pruebas.

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 21)

```
biblioteca_fastapi/  
├── api/  
│   └── libros.py  
├── services/  
│   └── libro_service.py  
├── schemas.py  
├── models/  
│   └── libro.py  
├── database.py  
└── main.py
```

(No hay archivos nuevos ni modificados)

Levanta la API

```
uvicorn main:app --reload
```

Endpoints disponibles:

- `http://localhost:8000/api/libros`
- `http://localhost:8000/docs`

2 Prueba DELETE en Postman

✓ Eliminar libro existente

Método: DELETE

URL: `http://localhost:8000/api/libros/1`

Respuesta esperada:

- Código **204 No Content**
- Sin body:

<vacío>

Esto confirma que el router devolvió correctamente 204.

3 Verificar que fue eliminado

Ahora prueba:

✓ Listado general:

GET

`http://localhost:8000/api/libros`

El libro con `id = 1` ya no debe aparecer.

✓ Consultar por ID:

GET

`http://localhost:8000/api/libros/1`

Respuesta esperada:

```
{
  "detail": "Libro con id 1 no encontrado"
}
```

Código: **404**

Esto confirma que la eliminación tuvo efecto tanto en BD como en API.

Caso de libro inexistente

Probar:

DELETE

`http://localhost:8000/api/libros/9999`

Respuesta esperada:

```
{  
  "detail": "Libro con id 9999 no encontrado"  
}
```

Código: **404**

Revisar la documentación en `/docs`

En Swagger UI debe aparecer:

- Endpoint DELETE `/api/libros/{id}`
- Sin request body
- Response 204
- Error 404 documentado

Puedes hacer las pruebas desde ahí directamente.

✓ Resultado esperado al terminar Fase 21

Tu API ahora:

- Elimina registros correctamente.
 - Responde con los códigos correctos.
 - Maneja errores de manera limpia.
 - Tiene CRUD completo.
-

¿Continuamos? Responde: **listo** para la Fase 22 (configuración de CORS).

Dijiste:

listo vamos a la fase 22

ChatGPT dijo:

FASE 22: CONFIGURACIÓN DE CORS

Por qué:

Los frontends modernos (Angular, React, Vite) corren en **puertos diferentes** al backend. Sin CORS, el navegador **bloqueará las peticiones** por seguridad.

En esta fase activaremos CORS en el backend:

- Permitiremos el acceso desde:
 - `http://localhost:4200` (Angular)
 - `http://localhost:5173` (React + Vite)
- Configuraremos `allow_methods`, `allow_headers`, `allow_credentials`.
- Modificaremos **main.py** marcando lo nuevo con # [NUEVO].

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 22)

```
biblioteca_fastapi/  
├── main.py          (MODIFICADO)  
├── api/  
│   └── libros.py  
├── services/  
│   └── libro_service.py  
├── schemas.py  
├── models/  
│   └── libro.py  
├── database.py  
└── .env
```

Archivo (MODIFICADO): **main.py**

Ruta:

- `C:\Cursos\FastAPI\biblioteca_fastapi\main.py`

Archivo **completo**, marcando solo las secciones nuevas # [NUEVO]:


```

# main.py
# Punto de entrada de la aplicación FastAPI.

from fastapi import FastAPI
from api.libros import router as libros_router
from fastapi.middleware.cors import CORSMiddleware # [NUEVO]

app = FastAPI(
    title="API Biblioteca Personal",
    description="Backend REST con FastAPI y MySQL para administrar libros personales.",
    version="1.0.0"
)

# -----
# Configuración de CORS # [NUEVO]
# -----
origins = [ # [NUEVO]
    "http://localhost:4200", # Angular # [NUEVO]
    "http://localhost:5173", # React Vite # [NUEVO]
] # [NUEVO]

app.add_middleware( # [NUEVO]
    CORSMiddleware, # [NUEVO]
    allow_origins=origins, # [NUEVO]
    allow_credentials=True, # [NUEVO]
    allow_methods=["*"], # [NUEVO]
    allow_headers=["*"], # [NUEVO]
)

# -----
# Endpoint de prueba inicial
# -----
@app.get("/")
def home():
    """
    Endpoint inicial para verificar que la API está funcionando.
    """
    return {"mensaje": "API de Biblioteca funcionando correctamente"}

# -----
# Incluir router de libros
# -----
app.include_router(libros_router)

```



Explicación de CORS

✓ ¿Qué es CORS?

Mecanismo de seguridad del navegador que controla qué dominios pueden llamar a tu API.

✓ ¿Por qué es necesario?

El backend corre en:

```
http://localhost:8000
```

Angular corre en:

```
http://localhost:4200
```

React (Vite) corre en:

```
http://localhost:5173
```

Dominios/puertos diferentes → sin CORS, el navegador bloquea todo.

✓ Opciones configuradas

- `allow_origins`: lista de orígenes permitidos.
- `allow_methods=["*"]`: permite todos los métodos (GET, POST, PUT, DELETE).
- `allow_headers=["*"]`: permite cualquier header.
- `allow_credentials=True`: permite cookies o headers de autenticación (si existieran).

Tu API ya está completamente lista para ser consumida desde Angular y React sin errores de CORS.

¿Continuamos? Responde: **listo** para la Fase 23 (cambio al puerto 8080 + pruebas finales).

Dijiste:

listo vamos a la fase 23

ChatGPT dijo:



FASE 23: CAMBIO AL PUERTO 8080 Y REPASO FINAL DE PRUEBAS

Por qué:

Hasta ahora hemos ejecutado FastAPI en el puerto predeterminado **8000**, pero nuestro backend final deberá estar disponible en:

```
http://localhost:8080/api/libros
```

En esta fase aprenderás:

- Cómo ejecutar FastAPI en el puerto **8080**.
- Cuáles comandos usar.

- Cómo verificar que todo el CRUD sigue funcionando correctamente.

En esta fase **no hay cambios de código**.

NUEVOS ARCHIVOS / MODIFICACIONES (Fase 23)

`biblioteca_fastapi/`

(No se crean ni modifican archivos)

1 Ejecutar FastAPI en el puerto 8080

En tu consola, dentro de la carpeta del proyecto:

```
uvicorn main:app --reload --port 8080
```

Esto levantará la API en:

- API raíz:
<http://localhost:8080/>
 - Documentación Swagger:
<http://localhost:8080/docs>
 - Endpoints CRUD de libros:
<http://localhost:8080/api/libros>
-

2 Verificar funcionamiento general en `/docs`

En:

<http://localhost:8080/docs>

Revisa que aparezcan todos los endpoints:

- GET `/api/libros`
- POST `/api/libros`
- GET `/api/libros/{id}`
- PUT `/api/libros/{id}`
- DELETE `/api/libros/{id}`

Todos deben estar agrupados bajo la etiqueta **Libros**.

3 Repaso completo del CRUD

Realiza las pruebas finales para confirmar que todo está funcionando.

✓ Crear un libro (POST)

POST <http://localhost:8080/api/libros>

Body JSON ejemplo:

```
{
  "titulo": "Cien Años de Soledad",
  "autor": "Gabriel García Márquez",
  "rating": 5
}
```

Debe devolver:

- Código: **201**
 - JSON con el libro creado
-

✓ Listar libros (GET)

GET <http://localhost:8080/api/libros>

Debe mostrar todos los libros agregados.

✓ Obtener libro por id (GET)

GET <http://localhost:8080/api/libros/1>

Debe devolver los detalles del libro.

✓ Actualizar libro (PUT)

PUT <http://localhost:8080/api/libros/1>

Body JSON:

```
{  
  "rating": 4  
}
```

Debe devolver:

- Código: **200**
 - JSON actualizado
-

✓ Eliminar libro (DELETE)

DELETE <http://localhost:8080/api/libros/1>

Debe devolver:

- Código: **204**
- Sin contenido

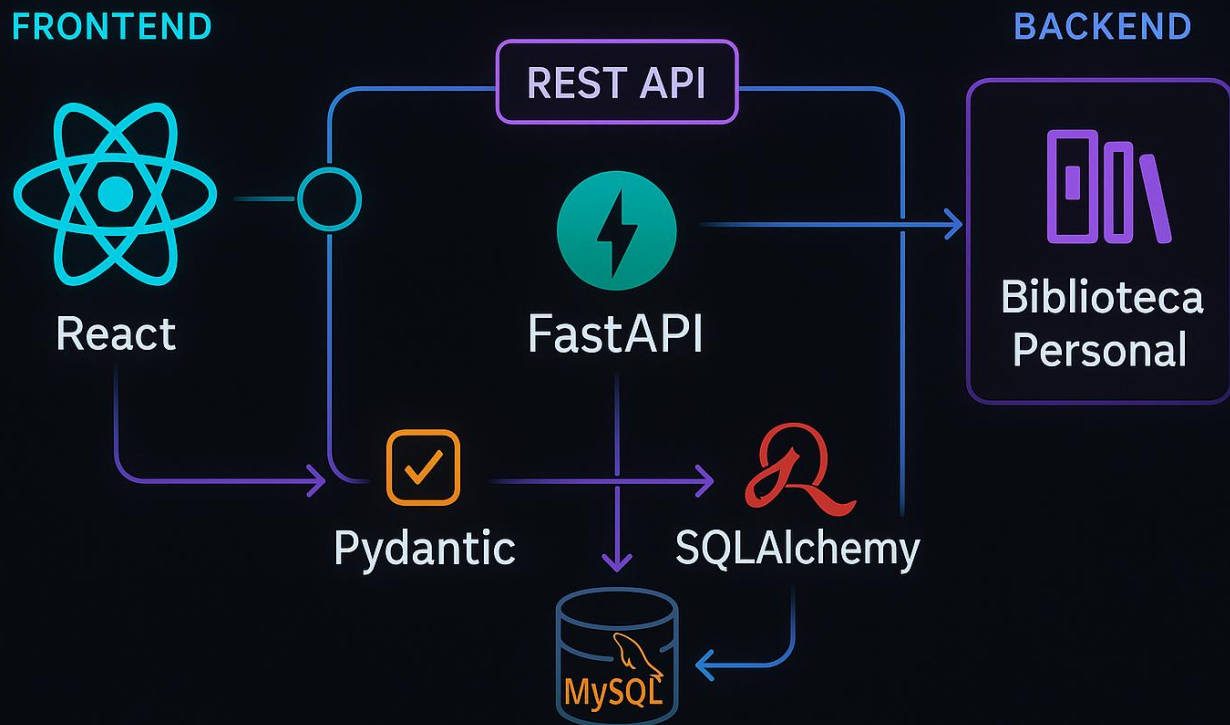
Luego validar:

GET [/api/libros/1](#) → 404

Confirmación final

Si todo funcionó correctamente:

- El backend funciona en el puerto **8080**.
 - Los endpoints responden correctamente.
 - La API está lista para conectarse con React o Angular.
-



🎉 Tu backend REST está oficialmente completo y funcional.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)